

AI 導入成效與效益分析報告

報告人：網站技術課 課長 EasonTsai | 回報主題：將「對話」轉化為「產能指標」

核心回報重點：

課內並非單純將 AI 視為聊天機器人，而是將其定位為已深入 **TAI-Builder** 開發體系核心的「數位勞動力」。透過 AI 協作矩陣，將 AI 深度嵌入至架構設計、系統分析、API 生產及自動化測試中，使課內在需求收斂與 API 產出效率提升約 **60%**，有效確保技術規格的一致性與長期可維護性。

四大維度：AI 協作應用分類與效率分析

1. 戰略與架構設計 (Macro Level) — 需求收斂與決策加速

具體應用內容

- **TAI-Builder 方法論演進**：定義 AI 協作矩陣，將開發流程標準化為「工程協議」。
- **系統架構評估**：針對大型專案（如官網智慧搜尋 PH1）進行精準技術選型與可行性分析。
- **開發模式轉型**：規劃與實踐從傳統 SDLC 轉型為 AI 驅動的 Vibe Coding 模式。

效率提升指標 (Productivity Metrics)

- **決策週期縮短**：原本需數週討論的架構與風險評估，透過 AI 模擬對抗與論證，縮短至 **3-5 天**。
- **文件完整度**：產出規格書具備更強的邏輯邊界，顯著減少後期改動成本。

2. 系統分析與 API 工程 (Meso Level) — 規格設計與精準對接

具體應用內容

- **API 規格產出**：根據商業邏輯快速生成符合 Restful 規範的 API 文件、Swagger 定義。
- **逆向工程**：透過 AI 快速解析舊系統複雜邏輯，產出對應的系統流程圖與重構建議。
- **資料建模**：自動生成高品質資料庫 Schema 設計與明確的資料關聯說明。

效率提升指標 (Productivity Metrics)

- **開發前置作業**：API 定義與介面對接文件的撰寫速度大幅提升 **60% - 70%**。
- **精準度提升**：消除人為疏漏造成的規格不對稱，大幅降低前後端溝通校對時間。

3. 開發實作與自動化 (Micro Level) — 核心代碼與自動化流程

具體應用內容

- **代碼生成與重構**：善用 Claude Code、Cursor 撰寫 Boilerplate 代碼並優化現有代碼效能。
- **單元測試與驗收**：自動生成全面性的 Test Cases 與測試邏輯，保障產出品質。
- **CI/CD 與運維自動化**：快速撰寫部署腳本（如 Jenkinsfile/GitHub Actions）與環境設定。

效率提升指標 (Productivity Metrics)

- **編碼速度提升**：基礎代碼撰寫效率提升 **2-3 倍**，釋放開發者心智模型以專注於 20% 核心商業邏輯。
- **Bug 檢出率**：開發階段透過 AI 進行靜態掃描與自我審查，降低 **30%** 以上初級邏輯錯誤。

4. 團隊管理與人才發展 (Management Level) — 組織升級與知識傳承

具體應用內容

- **人才招聘與評核**：快速撰寫精準的 JD、面試評估表，以及新進同仁與實習生的考核計畫。
- **技術培訓標準化**：將高階同仁的 AI 協作經驗轉化為課內可複用的 SOP 開發範式。

效率提升指標 (Productivity Metrics)

- **管理庶務自動化**：績效考評、工作報告與教育訓練規劃撰寫時間減少 **50%**。
- **知識傳承加速**：縮短初級工程師 (Junior) 的學習曲線，加速其獨立作業能力。

總結彙報建議（主管對話摘要）

「我們並非單純使用 AI 來寫代碼，而是透過 **TAI-Builder 方法論**，將 AI 嵌入到『架構設計、系統分析、API 生產、自動化測試』的雙卷體系中。透過 AI 協作矩陣，課內在需求收斂與 API 產出的效率上提升了約 **60%**，並確保了技術規格的一致性與可維護性。」